Chapter - 1
Learning From Data

Aviral Janveja

1 Introduction

Let us look at the fundamentals of machine learning, using an example from the domain of finance. Suppose someone applies for a bank loan. Now, the bank must decide whether granting the loan is a good idea or not, since their goal is to maximize profit while minimizing risk. Naturally, the bank has no magic formula to predict whether a customer will make timely repayments or not. However, it is indeed possible to look at similar customers from the past and examine how their repayment behavior turned out to be.

The **key idea** here, is to use past data points to uncover an underlying pattern that connects customer information with their repayment behavior. So that, we can use it to predict the creditworthiness of fresh applicants. Using this example, let us now outline the main elements of the machine learning process :

1.1 The Underlying Pattern

As discussed above, in machine learning, the idea is to uncover the underlying pattern from a given set of data points. In the bank loan example, this underlying pattern is the relation that connects customer information with their repayment behavior. We know some pattern exists for sure, but it is **unknown** to us and we only get glimpses of it through the available data points.

Target Function

In this section, we will describe this underlying pattern mathematically. But first, let us first consider a simpler scenario. Suppose the bank employs the following rule for loan approval :

> Approve every applicant whose annual income is greater than or equal to Rs. 7 lakh, otherwise reject the application.

This is a simple rule with no uncertainty. Every input produces exactly one correct output. Such a relation can be easily represented via a regular mathematical function, formally called the **target function** :

$$f(\mathbf{x}) = \begin{cases} +1 & \text{if } x \geq 7 \\ -1 & \text{if } x < 7 \end{cases}$$

Considering annual income as the only input, for simplicity. If $\mathbf{x} = 9$, then $f(9) = +1$, indicating that the loan shall be approved, as per the above rule.

Target Distribution

However, we know that real world decision making is rarely this simple. Two applicants may have identical ages, salaries and employment histories, yet one eventually repays the loan while the other defaults. This can happen due to factors that are not captured in our data, such as job loss, unexpected medical expenses or simply the inherent uncertainty in human life. This leads us to an important observation :

> Most practical, real world scenarios that we will deal with in machine learning are probabilistic in nature.

Therefore, instead of assuming a fixed output for every input, we describe the underlying pattern in terms of a conditional probability distribution, formally called the **target distribution** :

$$P(y \mid \mathbf{x})$$

Which basically tells us the probabilities of all possible outputs y, for a given input $\mathbf{x}$. For example :

$$P(y = +1 \mid x = 9) = 0.80$$
$$P(y = -1 \mid x = 9) = 0.20$$

Meaning that an applicant with a salary of Rs. 9 lakhs per annum, has an 80% probability of repaying the loan and a 20% probability of defaulting.

It is also important to **note** that the target function is simply a special case of the more general concept of target distribution. For example, when the probability of a certain output is 1 and thereby 0 for all others, the target distribution collapses to a target function :

$$P(y \mid \mathbf{x}) = \begin{cases} 1 & \text{if } y = f(\mathbf{x}) \\ 0 & \text{if } y \neq f(\mathbf{x}) \end{cases}$$

Input Distribution

So far, we have discussed how the target distribution quantifies the likelihood of all possible outputs for a given input. The target distribution remains the core underlying pattern that we will try to capture. However, there is another important probabilistic aspect to our data :

> The probability of encountering a certain type of input versus another.

For example, suppose there is a bag full of balls, of various colors. If we pick a ball at random, then the composition of the bag itself, would naturally lead to a probability of picking one color ball versus the other, as illustrated by the image below :



Figure 1: Bag of Balls

Similarly, in our bank loan example, imagine that the bag of balls represents the population of loan applicants in a particular city, where each applicant is described by a set of input attributes, such as age, annual income, employment history, outstanding debt and so on.

Then, just like the bag's natural composition determines the probability of picking a certain color ball, every population will naturally and inherently have an underlying probability of different input types. For instance, applications from married couples earning Rs. 20–25 lakhs per annum are going to be much more common, compared to let us say, children in school or retired old people. In machine learning, this probability distribution on the input space, is formally called the **input distribution** :

$$P(\mathbf{x})$$

The input distribution, just like the target distribution remains unknown to us. The only requirement is, that our data points are **independently** generated from the same underlying distribution. In the bag of balls analogy, it means sampling with replacement, or equivalently treating each draw as an independent draw from the same underlying distribution.

This is a fairly natural requirement, because our data is a sample from a certain population and we ultimately want our machine learning model to perform well on fresh observations drawn from that very same population.

Joint Distribution

So, now we have two probability distributions. Input distribution describing how likely we are to encounter a particular input and target distribution, describing the probability of all outputs, given that input.

Together, these distributions describe how our data points are generated. First, an input $\mathbf{x}$ is drawn according to $P(\mathbf{x})$. Then, given that input, an output y is generated according to $P(y \mid \mathbf{x})$. Hence, giving us the **joint distribution** :

$$P(\mathbf{x})\ P(y \mid \mathbf{x}) = P(\mathbf{x}, y)$$

1.2 The Data Set

The data set is simply a collection of data points generated by the unknown joint distribution, as described above. In the loan approval example, each data point corresponds to a past customer and collecting N such data points gives us the required data set :

$$\mathbf{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), (\mathbf{x}_3, y_3)...(\mathbf{x}_N, y_N)\}$$

Where $\mathbf{x}_n$ is the application information of the nth customer, represented by an array of numbers as shown below :

$$\mathbf{x}_n = \begin{bmatrix} x_1 \\ x_2 \\ . \\ . \\ x_d \end{bmatrix}$$

Here x_1, x_2 and so on upto x_d are data attributes of a customer like age, annual income, years in residence, outstanding debt and so on. While the corresponding output :

$$y_n \in \{+1, -1\}$$

Represents whether that customer eventually turned out to be creditworthy or not. Now, the data set does not reveal the target distribution completely, but it provides us with examples using which we can attempt to approximate it. In other words, the data set acts as our limited window into the underlying pattern.

2 Our First Learning Model : The Perceptron

So far, we have examined the two fundamental ingredients of every machine learning process, the **underlying pattern** and the **data set**. The next question is :

> How do we use the data set to learn something useful about the target distribution ?

To answer this question, we introduce perceptron, a simple linear model. Let us suppose that for a particular customer :

$$P(y = +1 \mid \mathbf{x}_n) = 0.80$$
$$P(y = -1 \mid \mathbf{x}_n) = 0.20$$

The perceptron does not attempt to estimate these probabilities. Instead, it only predicts the final binary output. In this case, it would simply predict :

$$y = +1$$

Although, this is only a simplified approximation of the underlying probability distribution, it provides an excellent starting point for understanding how machine learning models work. More advanced learning models introduced later will attempt to capture the probability distribution more explicitly.

2.1 Hypothesis Function

The perceptron works by assigning a numerical weight to each input data attribute, such as age, income and so on. The weighted attributes are then summed together to produce a single score :

$$w_1x_1 + w_2x_2 + ... + w_dx_d$$

This weighted sum can be interpreted as a credit score assigned by the perceptron. The idea is pretty simple, if the credit score is greater than a certain threshold, the loan is approved, otherwise it is denied :

$$\begin{aligned} w_1x_1 + w_2x_2 + ... + w_dx_d &> \text{threshold} \quad \text{(Approve)} \\ w_1x_1 + w_2x_2 + ... + w_dx_d &\leq \text{threshold} \quad \text{(Deny)} \end{aligned}$$

For mathematical convenience, we rewrite the threshold as another weight :

$$w_0 = -\text{threshold}$$

along with an artificial attribute to go along with it :

$$x_0 = 1$$

Hence, the decision rule now becomes :

$$\begin{aligned} w_0x_0 + w_1x_1 + w_2x_2 + ... + w_dx_d &> 0 \quad \text{(Approve)} \\ w_0x_0 + w_1x_1 + w_2x_2 + ... + w_dx_d &\leq 0 \quad \text{(Deny)} \end{aligned}$$

Next, we write the above expression in vector notation to make it clean and concise :

$$w_0x_0 + w_1x_1 + .. + w_dx_d = \begin{bmatrix} w_0 & w_1 & . & . & w_d \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ . \\ . \\ x_d \end{bmatrix} = \mathbf{w^T x}$$

Since, the above formula outputs a real number that could be greater than, equal to or less than zero whereas our desired output is binary, we pass this value through the **sign** function, which outputs $+1$ if the value is greater than zero and -1 otherwise, hence giving us the final perceptron hypothesis function :

$$h(\mathbf{x}) = \text{sign}(\mathbf{w^T x})$$

Hypothesis Set

The mathematical form of the hypothesis function is now fixed. The only unknown quantities are the weights contained in the vector $\mathbf{w}$. Weights are real numbers and can therefore take infinitely many different values.

Each different choice of the weight vector thereby produces a different hypothesis. The collection of all such candidate hypotheses is called the hypothesis set, denoted by :

$$\mathbf{H} = \{h_1, h_2, h_3....\}$$

2.2 Perceptron Learning Algorithm

With the hypothesis function fixed, the task now is to determine the values of the weight vector :

$$\mathbf{w}$$

We begin by initializing the weights randomly, usually to zero. The perceptron learning algorithm then iterates through the data set, searching for **errors**. For a single data point $(\mathbf{x}, y)$, the **point-wise error** is denoted by :

$$e(h(\mathbf{x}), y) \implies h(\mathbf{x}_n) \neq y_n \implies \text{sign}(\mathbf{w}^{\mathbf{T}}\mathbf{x}_n) \neq y_n$$

That is, when our hypothesis function disagrees with the observed output, on a particular data point, hence making it a misclassified point. Taking average of the point-wise errors over the entire data set, gives us the total **in-sample error** :

$$E_{in}(h) = \frac{1}{N}\sum_{n=1}^{N} e(h(\mathbf{x}_n), y_n)$$

In order to minimize the in-sample error, the learning algorithm updates the weight vector whenever a misclassified point is found, by applying the following **update rule** :

$$\mathbf{w}_{new} = \mathbf{w} + y_n\mathbf{x}_n$$

The learning algorithm continues making such updates to the weight vector, until no misclassified points remain in the data set or another stopping criterion is reached. Thereby arriving at the final set of weights that yield the lowest in-sample error, thus giving us our **final hypothesis** function :

$$E_{in} \approx 0 \implies h_f(\mathbf{x})$$

3 True Learning

However, remember that the goal in machine learning is not about performing well on the available data set, but to perform well on fresh unseen examples generated from the same joint distribution. That is, to have a low **out-of-sample error** :

$$E_{out}(h) = \mathbb{E}_{(\mathbf{x},y)}[e(h(\mathbf{x}), y)]$$

Which represents the **expected error** over the underlying joint distribution. Since the joint distribution is unknown, out-of-sample error cannot be computed or minimized directly. Instead, the learning model minimizes the in-sample error, expecting that the resulting final hypothesis will in turn achieve a low out-of-sample error as well, that is :

$$E_{in} \approx 0 \quad \text{and} \quad E_{out} \approx E_{in}$$

But, this raises an important question. How can we guarantee that a hypothesis that has a low in-sample error, will also achieve a low out-of-sample error?

3.1 Hoeffding Inequality

In order to tackle the fundamental question raised above, let us continue with our bank loan example and frame it as a simple probability experiment. Think of the applicants in a city as marbles in a pot. A marble is colored **green** if our hypothesis function classifies that applicant correctly, and **red** if we make an error. Our data set thus, corresponds to a sample drawn from that very same pot.

In line with the above analogy, the in-sample error for our current hypothesis function is represented by the fraction of red marbles in our sample. Meanwhile, the out-of-sample error corresponds to the unknown fraction of red marbles inside the pot, as shown below :

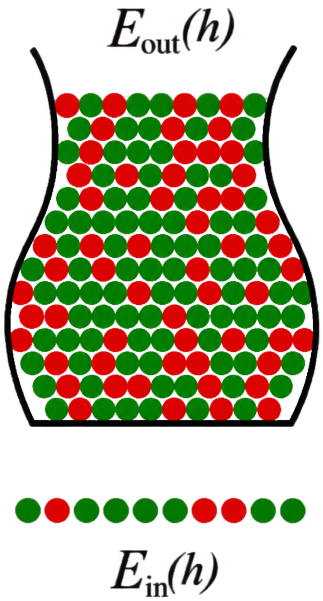


Figure 2: Pot of Marbles

Now, let us say a specific pot, representing a specific hypothesis, consists of 100 marbles, 90 green and 10 red, and suppose we randomly pick a sample of 10 marbles out of that pot **independently**. Meaning, we replace the marble after every pick, so that the next pick remains independent and the underlying probability distribution remains **identical**. Then, is it possible that all or most of those 10 marbles turn out to be red?

Well, it is **possible**, but is it **probable**? Not really.

Computing the exact probability shows that the chances of getting 5 or more reds in a sample of 10, collected from that particular pot, is only about 0.16%, which means that picking a mostly red sample from a mostly green pot is highly unlikely. In fact, as per the laws of probability, we can expect the fraction of red marbles in the sample to approximately reflect the fraction of red marbles in the pot, as the sample size grows larger.

The above example shows that, as per the laws of probability, we can expect $E_{in}(h)$ and $E_{out}(h)$ to track each other, with a large enough sample-size.

This probabilistic understanding is formalized by the **Hoeffding Inequality** as follows :

$$P[\ |E_{in}(h) - E_{out}(h)| > \epsilon\ ] \leq 2e^{-2\epsilon^2 N}$$

In simple words :

> The probability of E_{in} and E_{out} differing from each other will be low, if we accept a reasonable approximation and have a lot of data points.

Let us clarify the terms in the above inequality :

- $E_{in}(h)$: In-sample error of a hypothesis.
- $E_{out}(h)$: Unknown out-of-sample performance of a hypothesis.
- ϵ : The tolerance value, which is the maximum acceptable difference between E_{in} and E_{out}.
- N : Number of available data points.

It can be observed from the above expression that a **large** N makes the probabilistic bound tighter, giving us more confidence for out-of-sample performance, whereas a **small** ϵ makes the bound looser, since such precision is harder to guarantee.

As discussed in the section on input distribution, as long as our current data set (training date) and future examples (test data) are independently drawn from the same underlying distribution, the answer is yes.

4 Appendix

4.1 Update Rule Justification

In this section, we examine how the perceptron update rule actually nudges the classifier in the right direction at each misclassified point. If a point $(\mathbf{x}_n, y_n)$ is misclassified, it means :

$$\text{sign}(\mathbf{w}^\mathbf{T}\mathbf{x}_n) \neq y_n$$

Accordingly, the perceptron updates the weight vector as follows :

$$\mathbf{w}_{new} = \mathbf{w} + y_n\mathbf{x}_n$$

Substituting the updated weight vector into the linear score gives :

$$\mathbf{w}_{new}^\mathbf{T}\mathbf{x}_n = (\mathbf{w} + y_n\mathbf{x}_n)^\mathbf{T}\mathbf{x}_n$$

Expanding the right-hand side, we get :

$$\mathbf{w}_{new}^\mathbf{T}\mathbf{x}_n = \mathbf{w}^\mathbf{T}\mathbf{x}_n + y_n\mathbf{x}_n^\mathbf{T}\mathbf{x}_n$$

This is equivalent to :

$$\mathbf{w}_{new}^\mathbf{T}\mathbf{x}_n = \mathbf{w}^\mathbf{T}\mathbf{x}_n + y_n||\mathbf{x}_n||^2$$

Now, for a misclassified point, where the observed output is :

$$y_n = +1$$

Whereas our hypothesis function predicted :

$$\text{sign}(\mathbf{w^T x}_n) = -1 \implies \mathbf{w^T x}_n \leq 0$$

After the update, we get :

$$\mathbf{w}_{new}^{\mathbf{T}} \mathbf{x}_n = \mathbf{w^T x}_n + ||\mathbf{x}_n||^2$$

Since,

$$||\mathbf{x}_n||^2 > 0$$

The update increases the score assigned to that particular customer. This might not lead to an immediate sign correction, but it is clearly pushing it towards the correct side.

Similarly, when :

$$y_n = -1$$

And our hypothesis function predicts :

$$\text{sign}(\mathbf{w^T x}_n) = +1 \implies \mathbf{w^T x}_n > 0$$

The update gives us :

$$\mathbf{w}_{new}^{\mathbf{T}} \mathbf{x}_n = \mathbf{w^T x}_n - ||\mathbf{x}_n||^2$$

Hence, showing that each update to the weight vector nudges the perceptron in the required direction, improving its performance on that particular data-point.

4.2 Geometric Interpretation

Next, we examine the perceptron update rule from the lens of vector geometry. The perceptron hypothesis function classifies the data points as follows :

$$h(\mathbf{x}) = \text{sign}(\mathbf{w^T x})$$

The vector multiplication inside the sign function, can be written in the dot product notation as follows :

$$\mathbf{w^T x} = |\mathbf{w}| \, |\mathbf{x}| \, cos\theta$$

Consider an example, where the true output is $y = +1$, but our current hypothesis function misclassifies it as -1, as shown in the image below :

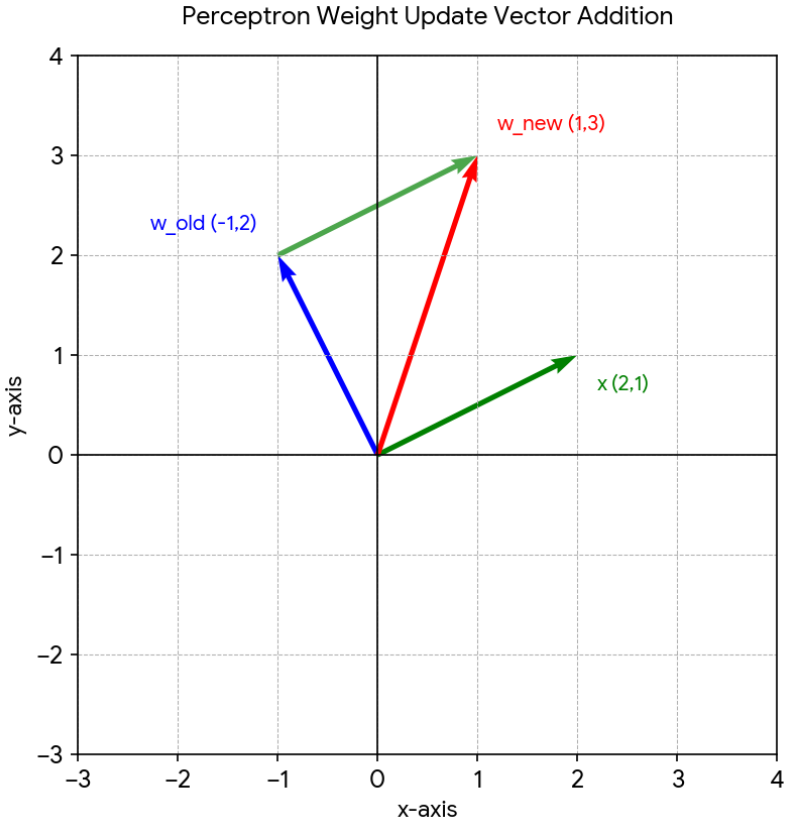


Figure 3: Vector Addition Geometry

Since,

$$\text{sign}(\mathbf{w}^{\mathbf{T}}\mathbf{x}) = -1 \implies \mathbf{w}^{\mathbf{T}}\mathbf{x} \leq 0 \implies |\mathbf{w}|\,|\mathbf{x}|\,cos\theta \leq 0$$

Which makes sense because the angle between $\mathbf{w}_{old}$ and $\mathbf{x}$ is between $90°$ and $180°$ and $\cos\theta$ is less than or equal to zero in this range. Since the above point is misclassified, as per the update rule, we update the weight vector as follows :

$$\mathbf{w}_{new} = \mathbf{w}_{old} + y\mathbf{x} \implies \mathbf{w}_{new} = \mathbf{w}_{old} + \mathbf{x}$$

The effect of this update on the angle between $\mathbf{w}_{new}$ and $\mathbf{x}$ is visible in the above image. The angle is now acute and hence the dot product will be now be positive, since $cos\theta > 0$, when the angle is between $0°$ and $90°$. So now :

$$|\mathbf{w}|\,|\mathbf{x}|\,cos\theta > 0 \implies \text{sign}(\mathbf{w}^{\mathbf{T}}\mathbf{x}) = +1 = y$$

Hence correcting the misclassified point. In general, while a single update might not correct the classification of a data point immediately, as shown in the above example, it clearly moves $\mathbf{w}_{new}$ in the required direction with respect to $\mathbf{x}$, such that the classification eventually becomes correct, possibly after multiple updates.

4.3 Convergence Theorem

The convergence theorem for perceptron guarantees that if the data set is **linearly separable**, repeated updates to the weight vector via the update rule will eventually classify all points correctly.

Data set being linearly separable simply means that, when the data points are plotted on a graph, it is possible to draw a straight-line that completely separates them into two distinct classes.

> This theorem is stated here without proof, as the proof is beyond the scope and requirement of this chapter.

5 References

1. CalTech Machine Learning Course. Learning From Data (CS156) : Lecture 1 - The Learning Problem.
2. CalTech Machine Learning Course. Learning From Data (CS156) : Lecture 2 - Is Learning Feasible?
3. CalTech Machine Learning Course. Learning From Data (CS156) : Lecture 4 - Error and Noise.